

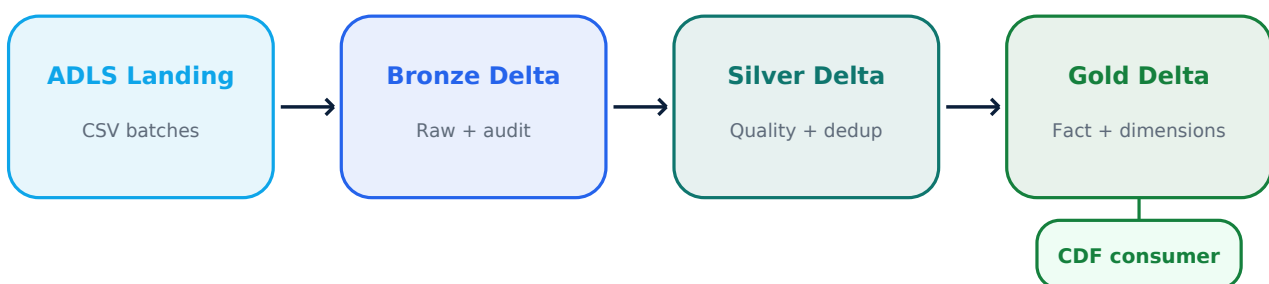
POC-02

Azure Databricks Medallion Lakehouse with Incremental CDC

Complete manual Azure setup, Terraform deployment, execution, validation, troubleshooting, screenshot evidence, and cleanup guide

Incremental Medallion flow

Auto Loader checkpoints prevent old files from being reprocessed; Delta MERGE applies inserts and updates.



Completion status

The evidence supplied for this guide shows successful Phase 1 and Phase 2 Databricks jobs, validated Bronze/Silver/Gold and quarantine data, SCD2 output, and successful Terraform cleanup of 10 resources.

Prepared from: POC_02_DATABRICKS_MEDALLION_CDC.md, step_terraform.md, steps_run_pocs2.txt, and 17 supplied screenshots.

How to use this guide

This document supports two valid execution paths. Choose one path for infrastructure creation, then use the shared Phase 1, Phase 2, validation, and cleanup sections.

Path	Use it when
Path A - Manual Azure Portal	Best for learning where every Azure and Databricks setting lives. Follow Chapters 3-8.
Path B - Terraform-first	Best for repeatable creation and destruction. Follow Chapters 9-12.
Shared execution	Both paths run the same notebooks, data quality rules, Delta MERGE, SCD1/SCD2, CDF, and SQL validation.

Important scope clarification

Azure SQL Server, Azure SQL Database, SQL firewall rules, Azure Data Factory, and an ADF pipeline are not used by POC-02. This POC uses ADLS Gen2, Azure Databricks, Unity Catalog, Databricks Jobs, PySpark, Delta Lake, Auto Loader, MERGE, SCD, and CDF. Creating SQL/ADF resources would add cost and would belong to the separate secure ADF-ADLS-SQL POC. This guide therefore documents every manual resource actually required for POC-02.

Guide map

Section	Contents
1-2	Objective, architecture, prerequisites, naming, and expected result
3-8	Manual Azure Portal and Databricks setup, data upload, jobs, and validation
9-12	Terraform configuration, apply, phase scripts, state safety, and destroy
13	Real errors encountered and the fixes that worked
14	Verification checklist and final observed counts
15	Cost controls, cleanup, GitHub safety, and interview summary
Appendix	All supplied screenshot evidence, including the one earlier POC-01 Terraform reference image

1. Objective and deliverables

Build a Bronze/Silver/Gold Delta Lake lakehouse on Azure Databricks and demonstrate incremental ingestion, schema evolution, data quality, deduplication, upserts, SCD Type 1, SCD Type 2, and downstream incremental consumption through Delta Change Data Feed (CDF).

What the completed POC demonstrates

- Bronze: preserves source records and adds `_ingest_ts`, `_source_file`, and `_batch_id`.
- Auto Loader: discovers only new landing files and stores progress in separate checkpoint/schema paths.
- Silver: casts business types, applies quality rules, routes bad records to quarantine, and keeps the latest record per order ID.
- Gold: creates `fact_orders`, `dim_product`, and an SCD2 `dim_customer`.
- CDC/CDF: Delta MERGE applies changed source rows and CDF exposes changed fact rows to an audit consumer.
- Governance: Unity Catalog provides `catalog.schema.table` organization, permissions, metadata, and lineage.
- Operations: two Databricks jobs run Phase 1 and Phase 2; Terraform can destroy the lab afterward.

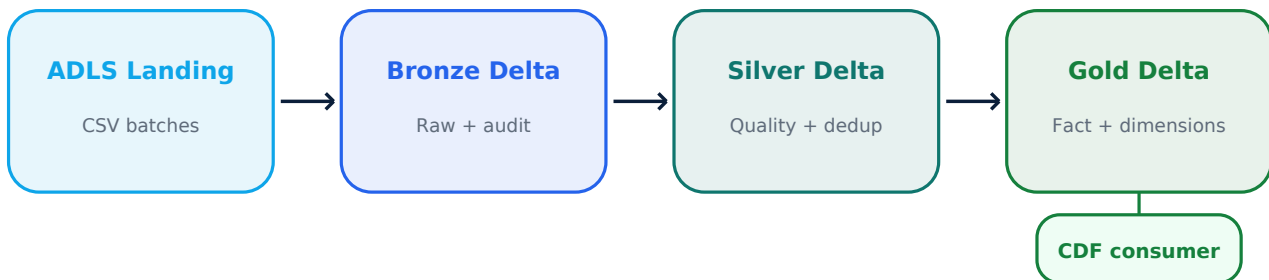
Expected project assets

Folder	Purpose
notebooks/	00_setup.py through 05_performance_governance.py
sample_data/phase1/	Initial orders and customers, including invalid and duplicate examples
sample_data/phase2/	New/updated rows plus <code>sales_channel</code> schema evolution and a customer city change
sql/	<code>validation_queries.sql</code>
terraform/	AzureRM + Databricks resources, variables, outputs, and provider configuration
cmd/	Numbered Windows CMD helpers from prerequisite check through destroy

2. Architecture, resources, and naming

Incremental Medallion flow

Auto Loader checkpoints prevent old files from being reprocessed; Delta MERGE applies inserts and updates.



Required resources

Resource	Example name	Purpose
Resource group	rg-azde-poc02	Lifecycle boundary for the lab
ADLS Gen2 account	stazdepoc02	Raw files, checkpoints, schema metadata, quarantine/external data
Container	poc02	Private container for this POC
Access Connector	ac-azde-poc02	System-assigned managed identity for secure storage access
Databricks workspace	dbw-azde-poc02	Notebooks, jobs, SQL, Unity Catalog, and compute
Storage credential	poc02_storage_cred	Unity Catalog credential backed by the Access Connector
External location	poc02_ext	abfss://poc02@.dfs.core.windows.net/
Catalog	azde_poc	Contains bronze, silver, gold, quarantine schemas

Storage layout

```
poc02/  
raw/orders/  
raw/customers/  
checkpoints/  
schema/  
quarantine/
```

Naming rule

Storage account names must be globally unique, lowercase, and alphanumeric. Keep the resource group, Databricks workspace, storage account, and Access Connector in one region unless your design explicitly requires otherwise.

3. Prerequisites and local setup

- 1 Azure subscription: use an account with enough permission to create the resource group, role assignments, Databricks workspace, and storage resources. Owner is simplest for a personal lab; Contributor alone cannot always create role assignments.
- 2 Azure CLI: install the CLI executable separately. The Python package `azure-identity` does not install the `az` command.
- 3 Terraform CLI: install Terraform 1.x and make sure its directory is on Windows PATH.
- 4 Project: extract the Terraform-enabled POC project to a short path without unusual characters.
- 5 Optional Databricks CLI: use it only if the supplied helper scripts or your workflow need it.

Run from Windows CMD

```
az --version
az login
az account list --output table
az account set --subscription "<subscription-id-or-name>"
az account show --output table
terraform -version
```

If az is not recognized

Close and reopen CMD after installing Azure CLI. Then run `where az`. A Poetry virtual environment can contain Azure Python SDK packages while the Azure CLI executable is still missing.

Pre-flight Azure checks

```
az account show --query "{subscription:name,id:id,tenant:tenantId,user:user.name}" --output table
az provider register --namespace Microsoft.Storage
az provider register --namespace Microsoft.Databricks
az provider register --namespace Microsoft.Authorization
```

- Confirm the selected subscription is the intended personal/lab subscription.
- Check regional VM quota before using a multi-node Databricks cluster.
- Never place Azure credentials, Databricks tokens, `terraform.tfstate`, or `terraform.tfvars` in a public GitHub repository.

4. Manual Azure Portal setup - infrastructure

4.1 Create the resource group

- 1 Open Azure Portal -> Resource groups -> Create.
- 2 Select the subscription, enter rg-azde-poc02, and choose the region used for this POC.
- 3 Select Review + create -> Create.
- 4 Verification: open the resource group and confirm it is empty before creating the remaining resources.

4.2 Create ADLS Gen2

- 1 Open Storage accounts -> Create. Select rg-azde-poc02.
- 2 Enter a globally unique name such as stazdepoc02fef00m; choose the same region and Standard/LRS for a small POC.
- 3 On Advanced, enable Hierarchical namespace. This is the setting that makes the account ADLS Gen2.
- 4 Keep public blob access disabled and use private container access.
- 5 Select Review + create -> Create. Verification: Overview shows Hierarchical namespace enabled.

4.3 Create container and folders

- 1 Open the storage account -> Data storage -> Containers -> + Container.
- 2 Create poc02 with Private access.
- 3 In the container, create raw/orders, raw/customers, checkpoints, schema, and quarantine. Empty directory markers are optional; uploads can create paths.

Evidence - resource groups created

Home > Resource Manager

Resource Manager | Resource groups

Export resource groups using Bicep or Terraform

How to manage changes with deployment tools?

Summarize my costs by service

Default Directory

Search

CreateManage viewRefreshExport to CSVOpen queryAssign tagsAdd to service groupGroup by none

Filter for any field...Subscription equals allLocation equals allAdd filter

☐	Name ↑	Subscription	Location
☐	NetworkWatcherRG	... Azure subscription 1	Central India
☐	rg-azde-poc02	... Azure subscription 1	Central India
☐	rg-azde-poc02-databricks-managed	... Azure subscription 1	Central India

Showing 1 - 3 of 3. Display count: auto

Give feedback

Add or remove favorites by pressing Ctr+L+Shift+F

Azure Portal resource-group list showing rg-azde-poc02 and the Databricks-managed resource group.

What this screenshot proves

The primary POC resource group and Databricks managed resource group exist in Central India.

Evidence - POC resource inventory

Home > Resource Manager | Resource groups

Resource Manager | Resource groups

Default Directory

Search

+ Create

Manage view

Resource Manager

All resources

Favorite resources

Recent resources

Resource groups

Tags

Organization

Tools

Deployments

Help

☐

Name ↑

☐

NetworkWatcherRG

☐

rg-azde-poc02

☐

rg-azde-poc02-databricks-manage

rg-azde-poc02

Resource group

Search

+ Create

Manage view

Delete resource group

Refresh

Group by none

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Generate Bicep code to duplicate this resource group.

+2

Essentials

JSON View

Resources

Recommendations

Filter for any field...

Type equals all

Location equals all

+ Add filter

☐

Name ↑

Type

Location

☐

ac-azde-poc02

...

Access Connector f...

Central India

☐

dbw-azde-poc02

...

Azure Databricks S...

Central India

☐

stazdepoc02fe00m

...

Storage account

Central India

rg-azde-poc02 contains the Access Connector, Azure Databricks workspace, and ADLS Gen2 storage account.

What this screenshot proves

The screenshot matches the intended minimal infrastructure: three top-level resources in the user-managed resource group.

5. Manual identity, IAM, and Databricks setup

5.1 Create the Access Connector

- 1 Search for Access Connector for Azure Databricks and select Create.
- 2 Use resource group rg-azde-poc02, name ac-azde-poc02, the same region, and System-assigned managed identity.
- 3 After deployment, open the connector and copy its complete Resource ID. Unity Catalog will use this ID.

5.2 Give the connector data-plane access

- 1 Open the storage account -> Access Control (IAM) -> Add -> Add role assignment.
- 2 Choose Storage Blob Data Contributor.
- 3 Assign access to Managed identity; choose Access Connector for Azure Databricks and select ac-azde-poc02.
- 4 Select Review + assign twice. Allow several minutes for RBAC propagation.

5.3 Optional: let your signed-in user upload files with Azure CLI

This is separate from the Access Connector permission. It is needed only when your own user runs `az storage fs file upload` with Entra authentication.

```
USER_OBJECT_ID=$(az ad signed-in-user show --query id -o tsv)
az role assignment create \
  --assignee-object-id "$USER_OBJECT_ID" \
  --assignee-principal-type User \
  --role "Storage Blob Data Contributor" \
  --scope "/subscriptions/<subscription-id>/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/<storage-account>"
```

Windows CMD note

CMD uses `for /f` instead of the Bash command substitution shown above. The safest manual sequence is: run `az ad signed-in-user show --query id -o tsv`, copy the returned object ID, and pass it to `--assignee-object-id`.

5.4 Create Azure Databricks workspace

- 1 Search Azure Databricks -> Create.
- 2 Use rg-azde-poc02, workspace name dbw-azde-poc02, and the same region.
- 3 Use Premium when Unity Catalog features require it in your account. Enable No Public IP when supported by your setup.
- 4 Create the workspace, then select Launch Workspace.

Evidence - storage IAM roles

Home > stazdepoc02fef00m

stazdepoc02fef00m | Access Control (IAM) ☆ ...

Storage account

Search

«

+ Add

Download role assignments

Edit columns

Refresh

Delete

Feedback

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Partner solutions

Resource visualizer

Data storage

Containers

Classic file shares

Queues

Tables

Security + networking

Data management

Check access

Role assignments

Roles

Deny assignments

Number of role assignments for this subscription ⓘ

4

5000

Search by name or email

Type : All

Role : All

Scope : All scopes

Group by : Role

All (3)

Job function roles (2)

Privileged administrator roles (1)

Name ↑↓

Type ↑↓

Role ↑↓

Scope ↑↓

Condition ↑↓

Owner (1)

IA

irshad alam

0cabca04-45aa-440e-9ce9-4b0...

User

Owner

Subscription (Inherited)

None

Storage Blob Data Contributor (2)

ac-azde-poc02

543c6a6f-619b-4de0-bf90-1f1...

Managed identity

Storage Blob Data Contributor

This resource

Add

IA

irshad alam

0cabca04-45aa-440e-9ce9-4b0...

User

Storage Blob Data Contributor

This resource

Add

Showing 1 - 3 of 3 results.

Add or remove favorites by pressing Ctrl+Shift+F

Storage account IAM shows Storage Blob Data Contributor for the Databricks Access Connector managed identity and for the signed-in user.

What this screenshot proves

Both runtime access and local CLI upload access were granted at the storage-account scope.

Evidence - Databricks workspace

The screenshot displays the Azure Databricks portal interface. At the top, the breadcrumb navigation shows 'All services > Azure Databricks'. The main header includes the 'Azure Databricks' logo and the workspace name 'dbw-azde-poc02'. Below the header, there are two tabs: 'Overview' (selected) and 'Essentials'. The 'Overview' tab shows a list of services including Activity log, Access control (IAM), Tags, Diagnose and solve problems, Resource visualizer, Settings, Monitoring, Automation, and Help. The 'Essentials' tab displays key information about the workspace, including its status (Active), resource group (rg-azde-poc02), location (Central India), subscription (Azure subscription 1), and subscription ID (9902e04b-fd65-40f4-86a2-1b09c1fe672c). It also shows the workspace type (Hybrid), managed resource group (rg-azde-poc02-databricks-managed), URL (https://adb-7405616874846049.9.azuredatabricks.net), pricing tier (Premium), and enable no public IP (Yes). A 'Launch Workspace' button is visible at the bottom right of the Essentials section. The left sidebar shows a list of workspaces, with 'dbw-azde-poc02' selected. The bottom of the page includes a footer with the text 'Add or remove favorites by pressing Ctrl+Shift+F'.

Azure Databricks resource overview for dbw-azde-poc02, Premium tier, managed by Terraform.

What this screenshot proves

The workspace, managed resource group, location, URL, and Terraform tags are visible.

6. Manual Unity Catalog and compute setup

6.1 Create a storage credential

- 1 In Databricks open Catalog -> Add -> Add a storage credential.
- 2 Name it poc02_storage_cred. Choose Azure Managed Identity.
- 3 Paste the Access Connector resource ID and create the credential.
- 4 Verification: validate the credential if the UI offers a validation action.

6.2 Create the external location

- 1 Open Catalog -> Add -> Add an external location.
- 2 Name: poc02_ext. URL: abfss://poc02@<storage-account>.dfs.core.windows.net/.
- 3 Choose poc02_storage_cred and create the location.
- 4 Grant the POC user/group the minimum required privileges: READ FILES, WRITE FILES, CREATE EXTERNAL TABLE, and CREATE MANAGED STORAGE as needed.

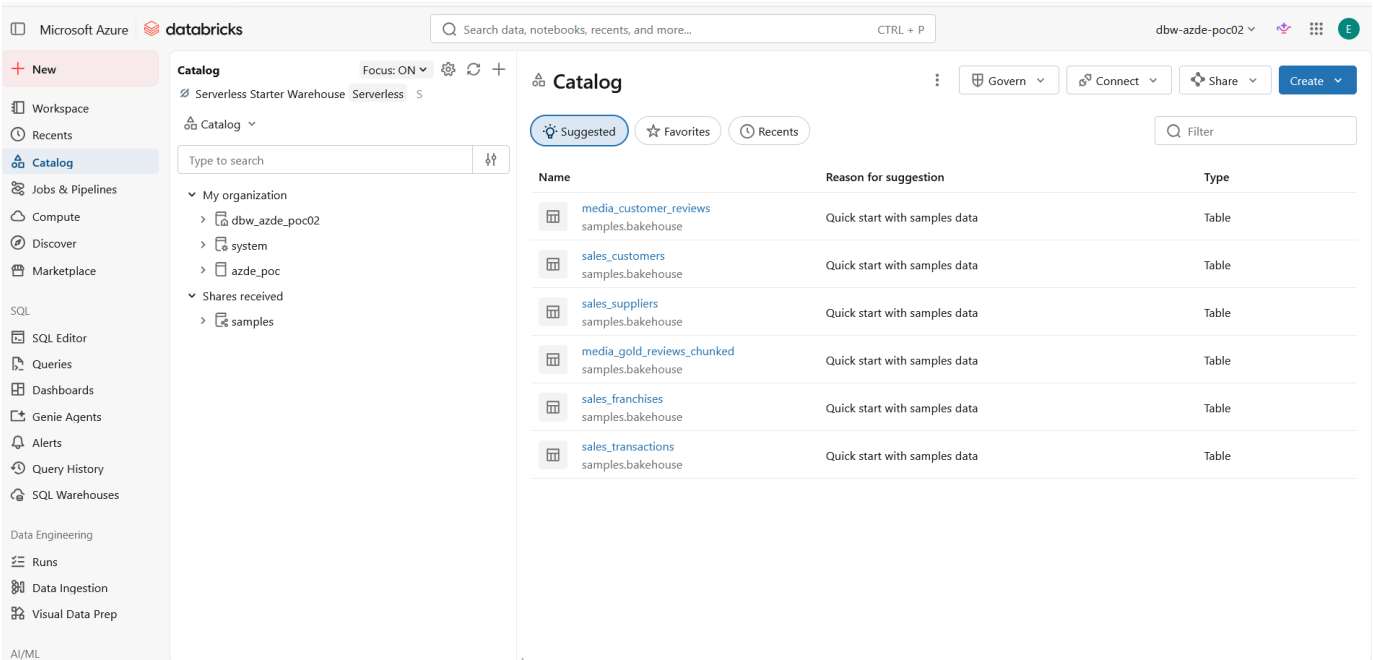
6.3 Create catalog and schemas

```
CREATE CATALOG IF NOT EXISTS azde_poc;  
CREATE SCHEMA IF NOT EXISTS azde_poc.bronze;  
CREATE SCHEMA IF NOT EXISTS azde_poc.silver;  
CREATE SCHEMA IF NOT EXISTS azde_poc.gold;  
CREATE SCHEMA IF NOT EXISTS azde_poc.quarantine;
```

6.4 Create low-cost compute

- 1 Prefer serverless compute when available, or create a small classic all-purpose/job compute.
- 2 For a constrained personal subscription, choose Single node, num_workers = 0, and 15-minute auto-termination.
- 3 If using Terraform job clusters, add the single-node Spark profile and ResourceClass tag shown in the troubleshooting chapter.
- 4 Verification: compute starts without AZURE_QUOTA_EXCEEDED_EXCEPTION and shows Unity Catalog data access.

Evidence - Unity Catalog available



Databricks Catalog Explorer showing the azde_poc catalog in the workspace.

What this screenshot proves

This early Catalog Explorer view confirms that Unity Catalog navigation and the POC catalog were available.

Evidence - catalog schemas managed by Terraform

The screenshot shows the Databricks Catalog Explorer interface. The left sidebar displays the navigation menu with options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, and Visual Data Prep. The main panel shows the 'azde_poc' catalog under the 'Serverless Starter Warehouse' and 'Serverless' configuration. The catalog description states: 'POC-02 Medallion catalog managed by Terraform'. Below the description, a table lists the schemas in the catalog:

Name	Owner	Created at
bronze	er.irshad26@outlook.com	Aug 31, 2026, 04:20 PM
gold	er.irshad26@outlook.com	Aug 31, 2026, 04:20 PM
information_schema	System user	Aug 31, 2026, 04:20 PM
quarantine	er.irshad26@outlook.com	Aug 31, 2026, 04:20 PM
silver	er.irshad26@outlook.com	Aug 31, 2026, 04:20 PM

azde_poc contains bronze, gold, quarantine, silver, and information_schema.

What this screenshot proves

The catalog description states it is managed by Terraform, and the four Medallion/quality schemas are present.

7. Import code, upload data, and run Phase 1

7.1 Import notebooks

- 1 Open Workspace and create Shared/POC_02_DATABRICKS_MEDALLION_CDC or use a Git Folder.
- 2 Import 00_setup.py, 01_bronze_ingest.py, 02_silver_quality.py, 03_gold_dimensions.py, 04_cdf_consumer.py, and 05_performance_governance.py.
- 3 Confirm the source format is Python and attach the intended compute only when running manually.

7.2 Upload Phase 1 only

```
az storage fs file upload --account-name <storage-account> --file-system poc02 --path raw/orders/orders_batch_001.csv --source sample_data\phase1\raw\orders\orders_batch_001.csv --auth-mode login --overwrite true
az storage fs file upload --account-name <storage-account> --file-system poc02 --path raw/customers/customers_batch_001.csv --source sample_data\phase1\raw\customers\customers_batch_001.csv --auth-mode login --overwrite true
```

Do not upload Phase 2 yet

The lab proves incremental processing only when Phase 1 is completed and validated before the second batch arrives.

7.3 Notebook parameters

Widget	Phase 1 value
storage_account	
container	poc02
catalog	azde_poc
batch_id	phase1

7.4 Run Phase 1 in order

Notebook	Expected action
00_setup.py	Creates/validates catalog, schemas, and path variables.
01_bronze_ingest.py	Auto Loader ingests orders/customers with metadata and checkpoints.
02_silver_quality.py	Casts, validates, quarantines, standardizes, and deduplicates.
03_gold_dimensions.py	Builds fact_orders, SCD1 dim_product, and SCD2 dim_customer; enables CDF on fact_orders.

7.5 Checkpoint test

- 1 Record the Bronze row counts.
- 2 Run 01_bronze_ingest.py again without uploading a new file.
- 3 The counts must stay unchanged. The checkpoint tracks processed input state; it is not business data.

Evidence - all notebooks imported

Microsoft Azure databricks

Search data, notebooks, recent, and more... CTRL + P

dbw-azde-poc02

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

Workspace

Home

Shared with me

Workspace

Repos

Shared

POC_02_DATABRICKS_MEDALLION_CDC

Users

er.irshad26@outlook.com

Groups

Favorites

Trash

Workspace > Shared >

POC_02_DATABRICKS_MEDALLION_CDC

Search

Type

Owner

Last modified

Name	Type	Owner	Created at	Last updated at
00_setup	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:...	Aug 31, 2026, 04:20 PM
01_bronze_ingest	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:...	Aug 31, 2026, 04:20 PM
02_silver_quality	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:...	Aug 31, 2026, 04:20 PM
03_gold_dimensions	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:...	Aug 31, 2026, 04:20 PM
04_cdf_consumer	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:...	Aug 31, 2026, 04:20 PM
05_performance_governance	Notebook	er.irshad26@outl...	Aug 31, 2026, 04:...	Aug 31, 2026, 04:20 PM

Workspace folder showing the six POC-02 notebooks.

What this screenshot proves

All notebook assets needed for setup, ingestion, quality, Gold, CDF, and performance/governance are available.

Evidence - first Phase 1 failure

The screenshot shows the Databricks job configuration interface for a job named 'job-294354939284236-run-832485362905592-poc02_small_cluster'. The left sidebar contains navigation options like 'New', 'Workspace', 'Recents', 'Catalog', 'Jobs & Pipelines', 'Compute', 'Discover', 'Marketplace', 'SQL', 'SQL Editor', 'Queries', 'Dashboards', 'Genie Agents', 'Alerts', 'Query History', 'SQL Warehouses', 'Data Engineering', 'Runs', 'Data Ingestion', and 'Visual Data Prep'. The main panel shows the job configuration with tabs for 'Configuration', 'Libraries', 'Event log', 'Spark UI', 'Driver logs', 'Metrics', 'Apps', and 'Spark compute UI - Master'. A red error message states: 'Cluster terminated due to error. Error code: AZURE_QUOTA_EXCEEDED_EXCEPTION. The VM size you are specifying is not available. [details] QuotaExceeded: Operation could not be completed ... Show details'. A yellow warning banner indicates: 'Based on your existing cloud quotas, your default compute size has been reduced. View or increase your quotas in Azure portal.' The configuration details are as follows:

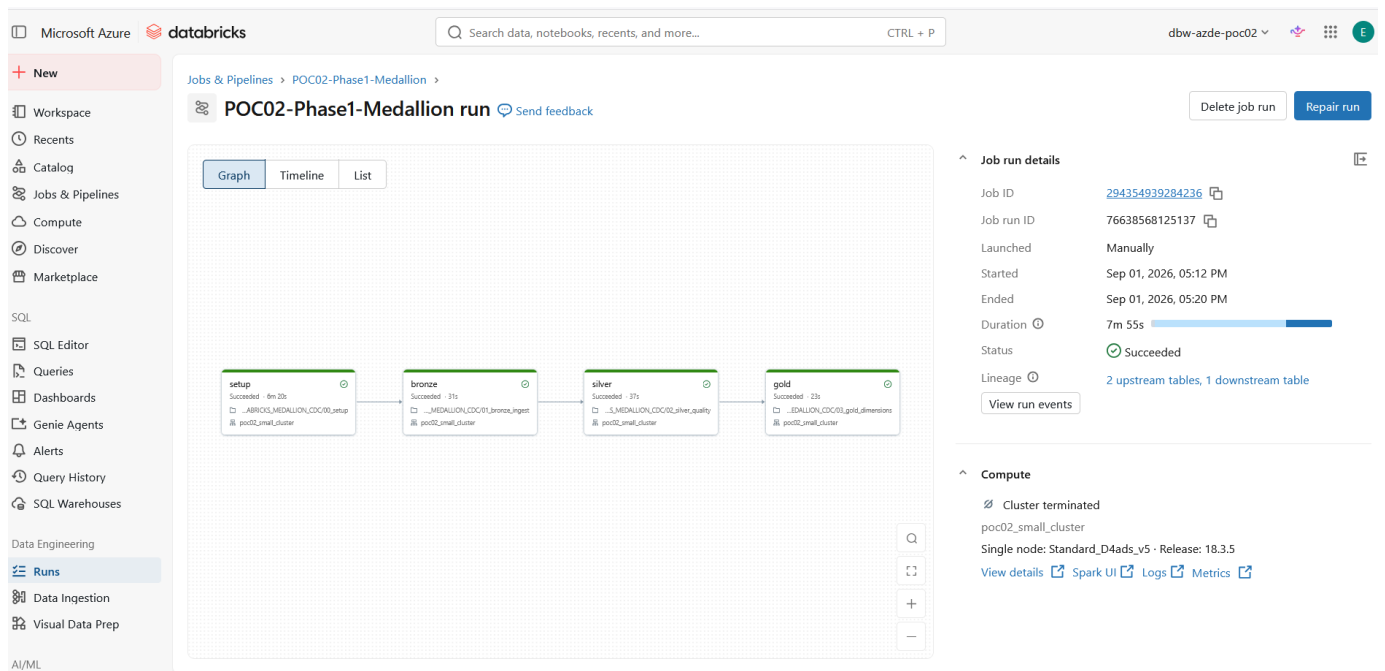
General	Performance	Summary
Compute name job-294354939284236-run-832485362905592-poc02_	Machine learning ☐ Databricks runtime 18 LTS (Scala 2.13, Spark 4.1.0)	Summary 16 GB Memory 4 Cores
Policy Unrestricted	Photon acceleration ☐	Data access Unity Catalog
Preferred worker type	Workers	Price 2 DBU/h

The first job compute terminated with AZURE_QUOTA_EXCEEDED_EXCEPTION.

What this screenshot proves

The original job configuration requested more regional vCPUs than the subscription quota permitted. This was an infrastructure sizing issue, not a PySpark code failure.

Evidence - Phase 1 succeeded after the compute fix



Databricks job graph showing setup, bronze, silver, and gold tasks succeeded.

What this screenshot proves

The full initial Medallion chain completed successfully in about eight minutes after switching to suitable single-node compute.

8. Run Phase 2, CDF, and validation

8.1 Upload Phase 2

```
az storage fs file upload --account-name <storage-account> --file-system poc02 --path
raw/orders/orders_batch_002_schema_evolution.csv --source
sample_data\phase2\raw\orders\orders_batch_002_schema_evolution.csv --auth-mode login --overwrite true
az storage fs file upload --account-name <storage-account> --file-system poc02 --path
raw/customers/customers_batch_002_customer_change.csv --source
sample_data\phase2\raw\customers\customers_batch_002_customer_change.csv --auth-mode login --overwrite true
```

8.2 Run Phase 2 tasks

- 1 Run Bronze ingestion with `batch_id = phase2`. Auto Loader sees only the new files.
- 2 The order file introduces nullable `sales_channel`. With `addNewColumns`, discovery can update schema metadata and require one restart. Rerun the failed task once after confirming the failure is the expected schema-evolution event.
- 3 Run Silver refresh and Gold merge. Validate that updates and inserts were applied by Delta MERGE.
- 4 Verify customer C002 has a closed Mumbai row and a current Bengaluru row in `dim_customer`.
- 5 Run `04_cdf_consumer.py` to write changed fact rows into `azde_poc.gold.fact_orders_changes_audit`.
- 6 Optionally run `05_performance_governance.py` and inspect query plans, partition counts, small-file considerations, comments, grants, and lineage.

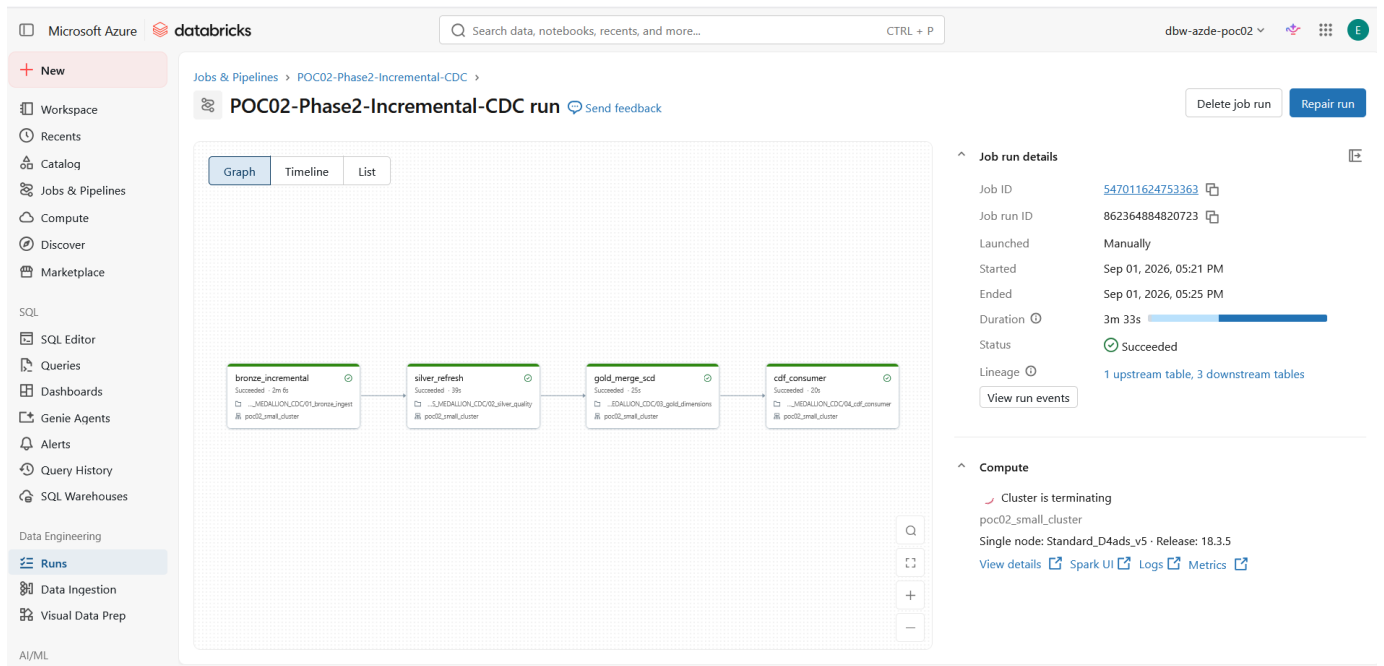
8.3 Core validation SQL

```
SELECT 'bronze.orders' AS table_name, COUNT(*) AS total_rows FROM azde_poc.bronze.orders
UNION ALL SELECT 'bronze.customers', COUNT(*) FROM azde_poc.bronze.customers
UNION ALL SELECT 'silver.orders', COUNT(*) FROM azde_poc.silver.orders
UNION ALL SELECT 'silver.customers', COUNT(*) FROM azde_poc.silver.customers
UNION ALL SELECT 'quarantine.orders_invalid', COUNT(*) FROM azde_poc.quarantine.orders_invalid
UNION ALL SELECT 'quarantine.customers_invalid', COUNT(*) FROM azde_poc.quarantine.customers_invalid;

SELECT error_reason, COUNT(*) AS record_count
FROM azde_poc.quarantine.orders_invalid
GROUP BY error_reason
ORDER BY error_reason;

SELECT customer_id, city, effective_from, effective_to, is_current
FROM azde_poc.gold.dim_customer
WHERE customer_id = 'C002'
ORDER BY effective_from;
```

Evidence - Phase 2 succeeded

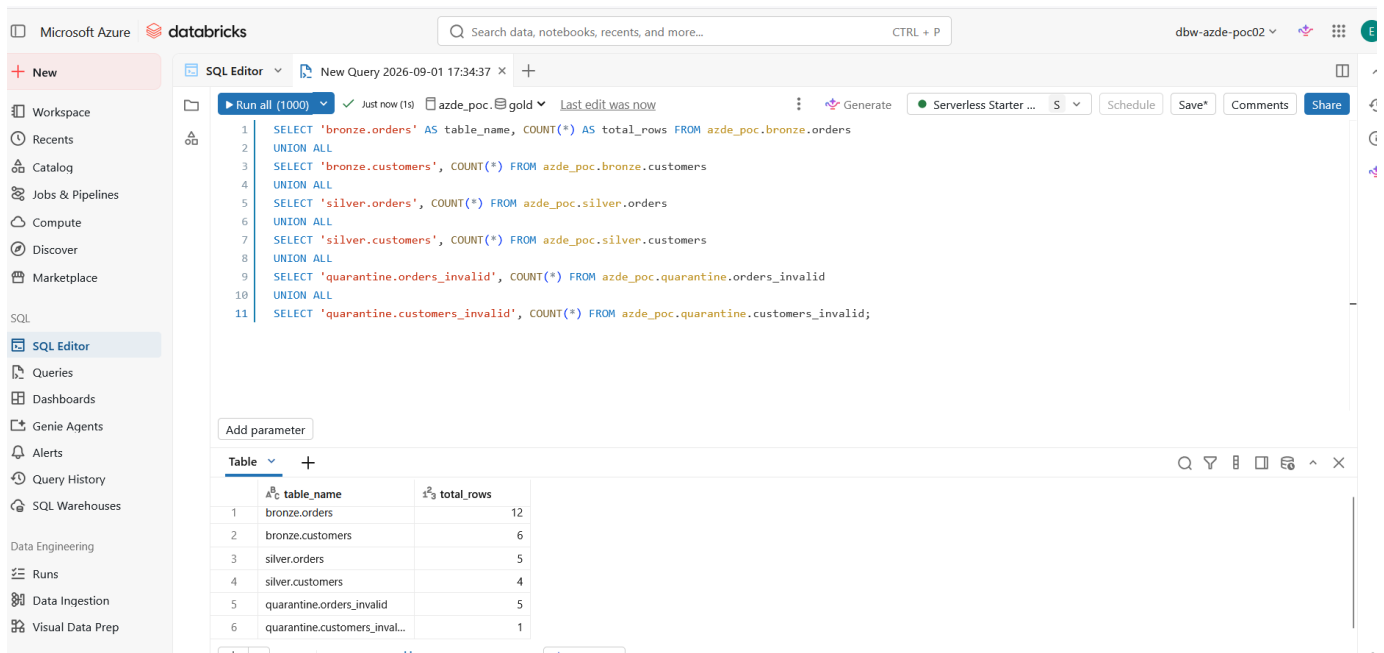


Databricks job graph showing bronze_incremental, silver_refresh, gold_merge_scd, and cdf_consumer succeeded.

What this screenshot proves

The incremental CDC path completed on a single-node Standard_D4ads_v5 job cluster.

Evidence - final layer and quarantine counts



The screenshot shows the Databricks SQL Editor interface. The SQL query is a UNION ALL statement that counts rows from various tables in the 'azde_poc' database. The query is as follows:

```
1 SELECT 'bronze.orders' AS table_name, COUNT(*) AS total_rows FROM azde_poc.bronze.orders
2 UNION ALL
3 SELECT 'bronze.customers', COUNT(*) FROM azde_poc.bronze.customers
4 UNION ALL
5 SELECT 'silver.orders', COUNT(*) FROM azde_poc.silver.orders
6 UNION ALL
7 SELECT 'silver.customers', COUNT(*) FROM azde_poc.silver.customers
8 UNION ALL
9 SELECT 'quarantine.orders_invalid', COUNT(*) FROM azde_poc.quarantine.orders_invalid
10 UNION ALL
11 SELECT 'quarantine.customers_invalid', COUNT(*) FROM azde_poc.quarantine.customers_invalid;
```

Below the query, the results are displayed in a table with two columns: 'table_name' and 'total_rows'.

Table	table_name	total_rows
1	bronze.orders	12
2	bronze.customers	6
3	silver.orders	5
4	silver.customers	4
5	quarantine.orders_invalid	5
6	quarantine.customers_invalid	1

Databricks SQL result across Bronze, Silver, and quarantine tables.

What this screenshot proves

The final screenshot shows Bronze orders 9, Bronze customers 6, Silver orders 5, Silver customers 4, quarantine orders 5, and quarantine customers 1.

Evidence - each order quality rule was exercised

The screenshot shows the Databricks SQL Editor interface. The query being executed is:

```
1 SELECT error_reason, COUNT(*) AS record_count
2 FROM azde_poc.quarantine.orders_invalid
3 GROUP BY error_reason;
```

The results are displayed in a table with 5 rows, showing one row for each of five order error reasons. The table has columns `error_reason` and `record_count`.

	error_reason	record_count
1	order_id is null/blank	1
2	quantity must be > 0	1
3	unit_price must be >= 0	1
4	status is not allowed	1
5	invalid order_ts	1

Below the table, it indicates 5 rows and 0.088s runtime. The interface also shows a sidebar with navigation options like Workspace, Recents, Catalog, and Jobs & Pipelines.

Quarantine grouping shows one row for each of five order error reasons.

What this screenshot proves

The POC explicitly tested null `order_id`, non-positive quantity, negative unit price, disallowed status, and invalid timestamp.

Evidence - SCD2 customer dimension

The screenshot shows the Databricks Catalog Explorer interface. On the left, the 'Catalog' sidebar is open, showing a tree view of the catalog structure. The 'gold' catalog is selected, and under it, the 'dim_customer' table is highlighted. The main panel displays the 'Sample Data' for the 'dim_customer' table. The table has four columns: 'customer_sk', 'customer_id', 'customer_name', and 'email'. The sample data shows four rows of customer information.

	customer_sk	customer_id	customer_name	email
1	8df003dcbfdd1fb4aef9540182f2a0646b922b3b0b4f091dbaa0650c4e9233aa	C001	Asha Khan	asha@example.com
2	db01268e5ae45ee63d627c348c52e91a52e2fc5e57596899840dcb7bc80d4bfc	C002	Ravi Mehta	ravi@example.com
3	e8fa9c2e0fb78856089cd4ceb5e5bd30feed7f7b8c4b1776314723ecb72311b	C003	Neha Singh	neha@example.com
4	f164a03ea2a6daed0d3fa13310af944e18bba38f810652a4d620c7ada0a3917d	C004	Imran Ali	imran@example.com

Catalog Explorer sample data for azde_poc.gold.dim_customer.

What this screenshot proves

The Gold catalog contains fact_orders, fact_orders_changes_audit, dim_product, and dim_customer; customer history rows are visible for validation.

9. Terraform-first deployment

Terraform should own every disposable infrastructure resource created for this lab. Do not mix Terraform creation with manual deletion, because that leaves stale state.

9.1 Configure variables

```
cd POC_02_DATABRICKS_MEDALLION_CDC_TERRAFORM_PROJECT
copy terraform\terraform.tfvars.example terraform\terraform.tfvars
notepad terraform\terraform.tfvars
```

Variable	Value / rule
subscription_id	The active Azure subscription ID
resource_group_name	rg-azde-poc02
location	centralindia or another region with available quota
storage_account_name	Globally unique lowercase alphanumeric name
databricks_workspace_name	dbw-azde-poc02
environment	lab or dev

9.2 Use numbered CMD scripts

```
cmd\00_check_prerequisites.cmd
cmd\01_configure_terraform.cmd
cmd\02_terraform_apply.cmd
```

9.3 Manual Terraform equivalent

```
cd terraform
terraform fmt -check
terraform init
terraform validate
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
terraform output
```

Review before apply

Confirm the plan contains only the intended POC resource group, ADLS account/container, Access Connector, RBAC, Databricks workspace, Unity Catalog objects, notebooks, and jobs. Never apply a plan that targets unrelated resources.

What Terraform creates

- Azure resource group, ADLS Gen2 account, and private file system/container.
- Databricks Access Connector and Storage Blob Data Contributor assignment.
- Azure Databricks workspace and workspace-level provider configuration.
- Unity Catalog storage credential, external location, catalog, and Bronze/Silver/Gold/quarantine schemas.
- Uploaded notebooks and Phase 1/Phase 2 Databricks jobs.
- Small single-node job compute definition suitable for the observed personal-subscription quota.

10. Terraform execution details and safe provider sequencing

Two provider contexts are involved

Provider	Responsibility
azurerem	Creates Azure resources such as resource group, storage, RBAC, Access Connector, and workspace.
databricks	Connects to the created workspace to configure Unity Catalog objects, notebooks, and jobs.

Recommended dependency order

- 1 Azure Resource Group -> Storage Account -> ADLS file system.
- 2 Access Connector -> Storage Blob Data Contributor RBAC assignment.
- 3 Databricks Workspace -> workspace URL output -> Databricks provider authentication.
- 4 Storage credential -> external location -> catalog -> schemas.
- 5 Notebook imports -> Phase 1 and Phase 2 jobs.

Single-node Terraform pattern

```
num_workers = 0

spark_conf = {
  "spark.databricks.cluster.profile" = "singleNode"
  "spark.master" = "local[*]"
}

custom_tags = {
  "ResourceClass" = "SingleNode"
}
```

State and generated files

- Commit .terraform.lock.hcl so provider versions remain reproducible.
- Do not commit .terraform/, terraform.tfstate*, tfplan, or a sensitive terraform.tfvars.
- For team use, move state to a protected Azure Storage backend with locking and restricted access. Local state is acceptable only for this isolated personal lab.
- After changing providers/modules/backend configuration, run `terraform init -upgrade` or `terraform init -reconfigure` only when the change requires it.

11. Terraform Phase 1 and Phase 2 runbook

Run Phase 1

```
cmd\03_upload_phase1.cmd  
cmd\04_open_phase1_job.cmd
```

- Wait for setup -> bronze -> silver -> gold to show Succeeded.
- If compute fails before Spark starts, inspect the cluster event log and quota, not notebook code.
- Rerun Bronze without new files and confirm row counts do not change.

Run Phase 2

```
cmd\05_upload_phase2.cmd  
cmd\06_open_phase2_job.cmd
```

- Wait for bronze_incremental -> silver_refresh -> gold_merge_scd -> cdf_consumer.
- If the first Bronze attempt stops after discovering sales_channel, rerun once to continue with the updated schema metadata.
- Use cmd\08_show_status.cmd and Databricks Job run details to confirm task status.

Validate Terraform state before cleanup

```
cd terraform  
terraform state list  
terraform show  
terraform output
```

Do not delete the resource group in Azure Portal first

If Terraform created the resources, let Terraform destroy them. A manual portal deletion can make the state inconsistent and complicate a repeat run.

12. Terraform destroy and post-cleanup checks

Preferred project script

```
cmd\07_destroy.cmd
```

The supplied safety wrapper asks for DESTROY before it performs the destroy. This reduces accidental deletion.

Manual equivalent

```
cd terraform
terraform plan -destroy -out=destroy.tfplan
terraform show destroy.tfplan
terraform apply destroy.tfplan
```

Post-destroy verification

```
az group exists --name rg-azde-poc02
az resource list --resource-group rg-azde-poc02 --output table
```

- Expected: the group no longer exists, or the resource list is empty while deletion finishes.
- Databricks-managed resource-group deletion can take several minutes after workspace destruction.
- Retain only source code, documentation, and sanitized screenshots. Remove local plans/state if they contain infrastructure metadata and are no longer needed.

Evidence - Terraform cleanup completed

```
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m10s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m20s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m30s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m40s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 02m50s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m00s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m10s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m20s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m30s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m40s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 03m50s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m00s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m10s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m20s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m30s elapsed]
azurerm_databricks_workspace.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-...t.Databricks/workspaces/dbw-azde-poc02, 04m40s elapsed]
azurerm_databricks_workspace.poc02: Destruction complete after 4m41s
azurerm_resource_group.poc02: Destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02]
azurerm_resource_group.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02, 00m10s elapsed]
azurerm_resource_group.poc02: Still destroying... [id=/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02, 00m20s elapsed]
azurerm_resource_group.poc02: Destruction complete after 24s
Destroy complete! Resources: 10 destroyed.
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT\terraform>
```

Terraform output showing Destroy complete: Resources: 10 destroyed.

What this screenshot proves

The workspace required several minutes to delete; Terraform completed the resource cleanup successfully.

Additional cleanup evidence - Azure CLI resource checks

```
Command Prompt - az groug x + v
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT\terraform>az group list --output
table
Name          Location    Status
-----
rg-azde-poc02 eastus      Succeeded

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT\terraform>az resource list --res
ource-group rg-azde-poc02 --output table
Name          ResourceGroup Location    Type                                Status
-----
stazdepoc02   rg-azde-poc02 eastus      Microsoft.Storage/storageAccounts Succeeded

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT\terraform>az group delete --name
rg-azde-poc02 --yes
- InProgress ..
```

Command Prompt showing az group list, az resource list, and a resource-group deletion command from an earlier cleanup attempt.

What this screenshot proves

This screenshot demonstrates the post-run Azure CLI checks. For Terraform-owned POC-02 resources, use terraform destroy as the authoritative cleanup path.

13. Real errors encountered and fixes

Symptom	Root cause	Fix
az is not recognized	Azure CLI executable was missing or PATH was not refreshed. azure-identity in Poetry is only an SDK package.	Install Azure CLI, reopen CMD, run where az, then az login.
Cannot find user or service principal in graph database	Role assignment used an email/UPN that Graph could not resolve.	Get the signed-in user object ID and use --assignee-object-id plus --assignee-principal-type User.
AZURE_QUOTA_EXCEEDED_EXCEPTION	Driver + worker requested 8 vCPUs while Central India quota was 4.	Use a single-node job cluster with num_workers=0, or request a quota increase.
New column detected by Auto Loader	Phase 2 added sales_channel under addNewColumns.	Allow schema metadata update, then rerun Bronze once; review the new nullable column.
Upload authorization failure	Signed-in CLI user did not have storage data-plane permission or RBAC had not propagated.	Assign Storage Blob Data Contributor to the user object ID at the storage account and wait several minutes.
Terraform asks for variable values	terraform.tfvars was missing/incomplete, or required variables had no defaults.	Create terraform.tfvars from the example and fill every required value before plan.

Exact role-assignment repair pattern

```
az ad signed-in-user show --query id -o tsv

az role assignment create --assignee-object-id "<returned-object-id>" --assignee-principal-type "User" --role "Storage Blob Data Contributor" --scope "/subscriptions/<subscription-id>/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/<storage-account>"
```

Evidence - storage permission issue fixed

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT>az role assignment create --assignee "er.irshad26@outlook.com" --role "Storage Blob Data Contributor" --scope "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/stazdepoc02fef00m"
Cannot find user or service principal in graph database for 'er.irshad26@outlook.com'. If the assignee is an appId, make sure the corresponding service principal is created with 'az ad sp create --id er.irshad26@outlook.com'.
```

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT>az ad signed-in-user show --query id -o tsv
0cabca04-45aa-440e-9ce9-4b0e258604fb
```

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT>az role assignment create --assignee-object-id "0cabca04-45aa-440e-9ce9-4b0e258604fb" --assignee-principal-type "User" --role "Storage Blob Data Contributor" --scope "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/stazdepoc02fef00m"
{
  "createdBy": "0cabca04-45aa-440e-9ce9-4b0e258604fb",
  "createdOn": "2026-08-31T10:46:02.603560+00:00",
  "description": "",
  "id": "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/stazdepoc02fef00m/providers/Microsoft.Authorization/roleAssignments/a9b48e68-8e9c-8ce2-2557-c92bf01c7ae9",
  "name": "a9b48e68-8e9c-8ce2-2557-c92bf01c7ae9",
  "principalId": "0cabca04-45aa-440e-9ce9-4b0e258604fb",
  "principalName": "er.irshad26_outlook.com#EXT#@erirshad26outlook.onmicrosoft.com",
  "principalType": "User",
  "resourceGroup": "rg-azde-poc02",
  "roleDefinitionId": "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/providers/Microsoft.Authorization/roleDefinitions/ba92f5b4-2d11-453d-a403-e96b0029c9fe",
  "roleDefinitionName": "Storage Blob Data Contributor",
  "scope": "/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/stazdepoc02fef00m",
  "type": "Microsoft.Authorization/roleAssignments",
  "updatedBy": "0cabca04-45aa-440e-9ce9-4b0e258604fb",
  "updatedOn": "2026-08-31T10:46:02.603560+00:00"
}
```

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_02_DATABRICKS_MEDALLION_CDC_PROJECT>
```

The email-based role assignment failed; resolving the signed-in user object ID and using --assignee-object-id succeeded.

What this screenshot proves

The command returned a Storage Blob Data Contributor role assignment for the signed-in user at storage-account scope.

Reference image - Terraform initialization

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_01_secure_batch_adf_adls_sql\infra\terraform>terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 4.0"...
- Installing hashicorp/azurerm v4.81.0...
- Installed hashicorp/azurerm v4.81.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_01_secure_batch_adf_adls_sql\infra\terraform>terraform plan
var.data_factory_name
  Globally unique Data Factory name.

Enter a value:
```

Earlier POC-01 screenshot showing successful terraform init and terraform plan prompting for a required variable.

What this screenshot proves

This image is included because it was supplied in the evidence ZIP. Its command path is POC-01, not POC-02; use it only as a reference for expected Terraform initialization output.

14. Final verification checklist

Observed final counts from the supplied SQL screenshot

Table	Observed rows
azde_poc.bronze.orders	9
azde_poc.bronze.customers	6
azde_poc.silver.orders	5
azde_poc.silver.customers	4
azde_poc.quarantine.orders_invalid	5
azde_poc.quarantine.customers_invalid	1

Quality-rule evidence

Quarantine reason	Observed rows
order_id is null/blank	1
quantity must be > 0	1
unit_price must be >= 0	1
status is not allowed	1
invalid order_ts	1

Completion checklist

- [PASS] Resource group, ADLS Gen2, Access Connector, IAM, and Databricks workspace were created.
- [PASS] Unity Catalog azde_poc and bronze/silver/gold/quarantine schemas exist.
- [PASS] Six notebooks were imported.
- [PASS] Phase 1 setup, Bronze, Silver, and Gold tasks succeeded.
- [PASS] Rerun behavior is protected by Auto Loader checkpoints.
- [PASS] Invalid rows are routed to quarantine with a reason.
- [PASS] Phase 2 incremental load, schema evolution, Gold MERGE/SCD, and CDF consumer succeeded.
- [PASS] Gold dim_customer preserves history with effective dates and is_current.
- [PASS] Final SQL validation results were captured.
- [PASS] Terraform reported 10 resources destroyed.

Completion statement

Based on the supplied execution notes and screenshots, POC-02 is complete. The strongest evidence is the successful Phase 1 and Phase 2 job graphs, the final SQL counts, the five quarantine-rule results, the Gold SCD2 table view, and the Terraform destroy output.

15. Operations, safety, and interview summary

Cost guardrails

- Keep the dataset small and use single-node or serverless compute.
- Set auto-termination to 15 minutes for classic compute.
- Terminate compute immediately after validation.
- Use Terraform destroy after the lab and confirm the Databricks managed resource group is removed.
- Do not create Azure SQL or ADF resources for this POC unless you intentionally design a separate integration extension.

Public GitHub safety

- Safe: Terraform source, .terraform.lock.hcl, sanitized tfvars.example, notebooks, sample data, SQL, README, and sanitized screenshots.
- Never commit: access tokens, client secrets, Azure CLI profile, terraform.tfstate*, tfplan files, private tfvars, or workspace tokens.
- Subscription IDs and object IDs are not passwords, but sanitize personal identifiers in public screenshots when they are unnecessary.

Interview-ready explanation

- Bronze vs Silver vs Gold: Bronze preserves source and audit metadata; Silver standardizes and validates; Gold provides business facts/dimensions.
- Checkpoint: state that tracks which files/offsets were processed; it is separate from business data.
- MERGE vs overwrite: MERGE applies key-based inserts/updates and supports incremental/idempotent designs; overwrite replaces a target scope.
- CDC vs CDF: CDC is the change-capture pattern; Delta CDF is a table feature that exposes row-level changes with change metadata.
- SCD1 vs SCD2: SCD1 keeps only current values; SCD2 closes the old row and inserts a new current row to retain history.
- Small files: they increase listing, scheduling, and metadata overhead; production designs compact files and avoid unnecessary partitions.
- Unity Catalog: centralizes object naming, permissions, ownership, metadata, and lineage.

Appendix A - Command quick reference

Create and run

```
cmd\00_check_prerequisites.cmd  
cmd\01_configure_terraform.cmd  
cmd\02_terraform_apply.cmd  
cmd\03_upload_phase1.cmd  
cmd\04_open_phase1_job.cmd  
cmd\05_upload_phase2.cmd  
cmd\06_open_phase2_job.cmd  
cmd\08_show_status.cmd
```

Destroy

```
cmd\07_destroy.cmd
```

Manual Terraform lifecycle

```
cd terraform  
terraform init  
terraform validate  
terraform plan -out=tfplan  
terraform apply tfplan  
terraform state list  
terraform plan -destroy -out=destroy.tfplan  
terraform apply destroy.tfplan
```

Useful Azure checks

```
az account show --output table  
az group show --name rg-azde-poc02 --output table  
az resource list --resource-group rg-azde-poc02 --output table  
az role assignment list --scope "/subscriptions/<subscription-id>/resourceGroups/rg-azde-poc02/providers/Microsoft.Storage/storageAccounts/<storage-account>" --output table
```

Appendix B - Source evidence inventory

Input	How it was used
POC_02_DATABRICKS_MEDALLION_CDC.md	Original scope, architecture, skills, validation, cleanup, and interview goals.
step_terraform.md	Manual Azure Portal setup, Unity Catalog steps, Phase 1/2 execution, and validation flow.
steps_run_pocs2.txt	Actual command sequence, final error history, quota root cause, and working fixes.
screenshot(1).zip	17 screenshots showing setup, failures, fixes, success, validation, and cleanup.

Evidence completeness

Every screenshot from the supplied ZIP is included in this document. Images were placed beside the relevant implementation step; the earlier POC-01 Terraform initialization screenshot is explicitly labeled as a reference image to avoid confusing it with POC-02.